Modern Database Systems

Seminar Assignment

submitted by:	Anh Thu Maria Bui
Registration Number:	1266909
	anh_thu_maria.bui@smail.th-koeln.de
submitted by:	Natalie Hußfeldt
Registration Number.:	11266912
	natalie.hussfeldt@smail.th-koeln.de

Master Digital Science SoSe 2024
Course : Modern Database Systems
Prof. Dr. Johann Schaible
Cologne, 03.07.2024

Contents

1 Introduction **1**
1.1 Description of the data . . . 1
1.2 Explanation of the use case . . . 1
1.3 Typical queries and their significance for the use case . . . 1

2 Database Selection **2**
2.1 Justification for the choice of database . . . 2
2.2 Data Modelling . . . 3
2.2.1 Modelling the data for PostgreSQL . . . 3
2.2.2 Modelling the data for Neo4j . . . 4

3 Performance results of the database queries **5**

4 Discussion of Results **7**

5 Theoretical Outlook **9**

6 Conclusion and Recommendations **10**

A Appendix **13**

List of Tables

1 Dataset Size . . . 1
2 Summary of Queries . . . 2
3 Transformation from Relational Tables to Graph Database Nodes and Relationships 5
4 Summary of Queries and Their Complexity . . . 15
5 Performance of Insert Queries on Relational and Graph Databases . . . 15
6 Performance of Recommendations Queries on Relational and Graph Databases . . . 17
7 Performance of Risk Students Queries on Relational and Graph Databases . . . 19
8 Performance of Common Courses Queries on Relational and Graph Databases . . . 20
9 Performance of Students in a Course Queries on Relational and Graph Databases . . . 22
10 Query 1: Results Recommendations, 9 rows total . . . 24
11 Query 2: Results At-Risk Students, 185 rows total . . . 24
12 Query 3: Results Common Courses, 294 rows total . . . 24
13 Query 4: Results Students in course, 295 rows total . . . 24

List of Figures

1 Relational database architecture . . . 3
2 Graph database architecture . . . 4
3 Performance Comparison . . . 6
4 Performance Writing Query 1 . . . 6
5 Performance Reading Query 1 . . . 7

6 Reading Query 2 7
7 Reading Query 3 7
8 Reading Query 4 7

1 Introduction

Universities place a priority on student success and pedagogical advancement, with a particular focus on the development of teaching skills. The use of data-driven methodologies is essential for the generation of actionable insights, particularly in the early identification of at-risk students and the recommendation of study materials [1]. By identifying these students at an early stage, institutions can not only enhance their graduation rates but also enrich the educational experience of their students [19].

This study compares the effectiveness of storing and managing university-related data using two different data models: a locally hosted relational PostgreSQL database (version 16.3) and an online graph database hosted by Neo4j AuraDB Free version "5.20-aura".

1.1 Description of the data

The Kaggle Open University Learning Analytics Dataset (OULAD) comprizes anonymized data about courses called modules, students and their interactions with the Virtual Learning Environment (VLE). The dataset consists of detailed records of students' interactions with an online learning platform, including their enrolment in different courses, use of learning materials and performance in assessments. It is designed to facilitate research on online learning behaviour and consists of multiple tables, which are stored in several files in CSV format and connected through unique identifiers. Table 1 illustrates the size of each CSV file. It should be noted that the dataset was initially reduced due to limited space in the NoSQL database. The CSV file `studentRegistration.csv` is not included in the list as it was not used in this study.

File name	**Size**
courses.csv	250 B
vle.csv	161 KB
assessments.csv	6 KB
studentInfo.csv	456 KB
studentAssessment.csv	2.2 MB
studentVle.csv	208.6 MB

Table 1: Dataset Size

1.2 Explanation of the use case

Key tasks include identifying students in specific courses, tracking their interactions with learning materials, and correlating these interactions with their grades. Queries analyse data to flag at-risk students and recommend personalised learning materials. This includes calculating average clicks on learning materials, comparing individual student activity to averages, and evaluating courses shared by students. The aim is to improve learning outcomes and prevent drop-outs through early intervention. Without efficient tracking and analysis, universities may struggle to improve teaching methods and manage resources effectively.

1.3 Typical queries and their significance for the use case

To compare the performance of PostgreSQL and Neo4j in handling different complexities and tasks, queries representing common educational operations and analytical tasks were selected. These

queries cover data insertion, student engagement analysis and course-specific data retrieval. While this study focuses on these queries, the use case may also require additional queries to delete or update data. The exact queries and their results tables can be found in the appendix A.

Table 2: Summary of Queries

Query	Description
Writing Query 1	This query inserts new students, courses, learning materials, and assessment records at the start of each term and updates throughout the term.
Reading Query 1	This query aggregates clicks on learning materials by filtering students enrolled in a specific course module based on enrolment year and grade thresholds. It reveals material usage patterns and provides insight into correlations between student engagement and academic performance in previous semesters. For example, the material clicked on by former top students in a particular course can be recommended.
Reading Query 2	The query identifies students with fewer clicks than the module average, suggesting they may need extra support to improve engagement and performance. The goal is to provide early alerts for students at risk by identifying those with lower than average engagement metrics.
Reading Query 3	This query aims to connect students by identifying those who share common courses. In a university with a wide range of courses, recommending study partners can help improve collaboration and academic performance.
Reading Query 4	This query identifies all students enrolled in a specific course and academic term. It enables the analysis of student engagement and the monitoring of performance across the entire class. By tracking course participation, one can gain a deeper understanding of the student demographic and improve learning outcomes.

2 Database Selection

Database selection is crucial in modern data management. Relational databases like PostgreSQL excel in handling structured data and ensuring ACID (Atomicity, Consistency, Isolation, Durability) compliance, ideal for transactional applications. However, the rapid growth of data from web technologies, social networks, and IoT demands scalable, flexible databases. NoSQL databases, including key-value, wide-column, document, and graph databases, prioritize scalability and flexibility over strict ACID compliance.[4]

For this study a locally hosted PostgreSQL 16.3 database and a Neo4j AuraDB Free version "5.20-aura" graph database were chosen. Neo4j uses property graphs to represent networks of entities and relationships, enriching connections with attributes and metadata for high performance and flexibility [6].

2.1 Justification for the choice of database

It can be argued that in real-world scenarios, data often exists as interconnected objects rather than isolated rows and columns. Consequently, relational databases struggle to represent highly connected data, due to their strict schema and reliance on complex JOIN operations. [20] In contrast, graph

databases such as Neo4j excel at handling these relationships in application domains such as social networks. This makes them ideal for the university use case, which involves a network of students, courses, grades and learning materials.[6]

The property graph model in Neo4j is particularly well-suited for this data as it allows entities (nodes) and their relationships (edges) to store additional attributes, such as grades or course details, directly within the graph structure. This model simplifies and optimizes queries involving path traversals and aggregations, such as identifying common courses or tracking student interactions with learning materials.[21, pp. 12, 13]. Graph databases excel in managing connected data structures compared to other NoSQL types like key-value or document-oriented databases, as they eliminate the need for complex joins [21, p. 15].

Graph databases offer greater flexibility to adapt to changes in data, such as adding new types of relationships or properties, which is essential in a dynamic educational environment. Adjustments such as adding new types of relationships or properties to nodes (e.g. introducing new types of learning materials or assessment types) are easier to accomplish in a graph database.[20] Neo4j's Cypher query language, with its intuitive syntax and graph-specific features, further enhances the ability to build expressive and efficient queries.[25] This capability makes Neo4j a good choice for managing the complex and evolving data structures typical in a university setting.

2.2 Data Modelling

In order to effectively compare execution times between relational (e.g. PostgreSQL) and NoSQL (e.g. Neo4j) databases for the use case, it is crucial to understand their individual data models. This section outlines the methodology used to develop these models, ensuring that database-specific rules are followed to optimise query performance and utilise the strengths of each database type.

2.2.1 Modelling the data for PostgreSQL

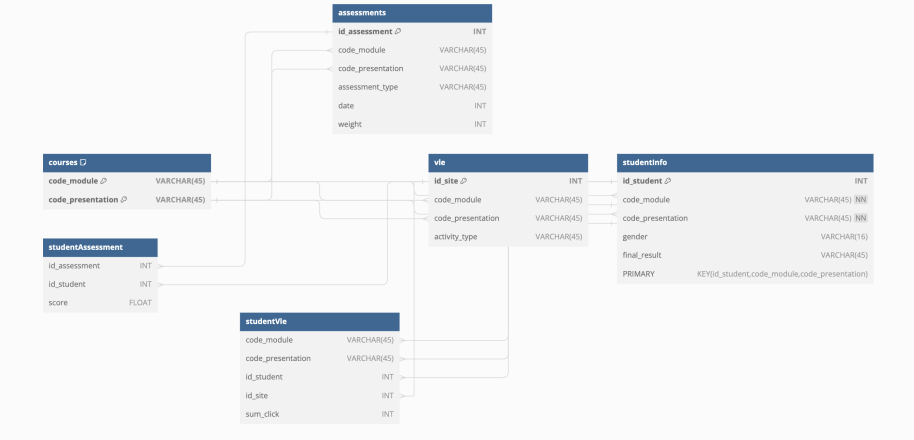

Figure 1: Relational database architecture

The OULAD data, already partitioned into various tables 1.1, stored across CSV files and linked by unique identifiers, aligns naturally with the schema of relational databases. Minor adjustments were

made to tailor the data specifically for the use case, such as removing the `studentRegistration` table and unnecessary columns like `studied_credits` and `disability` from `studentInfo` (see Figure 2).

Relational database structures follow normalisation guidelines to eliminate redundancy and anomalies. The First Normal Form (1NF) ensures all records have the same number of fields and eliminates repeating fields and groups. The Second Normal Form (2NF) requires that all attributes be functionally dependent on the entire primary key, not just part of it. The Third Normal Form (3NF) Specifies that no non-key attribute should depend on another non-key attribute. [9, 3] All tables in the schema conform to the Third Normal Form, ensuring that each non-key attribute is functionally dependent only on the primary key.

The focus of this study is on analysing data to identify at-risk students and provide personalised recommendations for learning materials. As the dataset grows, the size of the relational database relationships increases, which can affect the efficiency of search queries. This is because all tuples must be searched to match a particular condition. To improve performance in this context, indexes have been created on certain tables, such as the `studentInfo` table.[27, p. 14]

2.2.2 Modelling the data for Neo4j

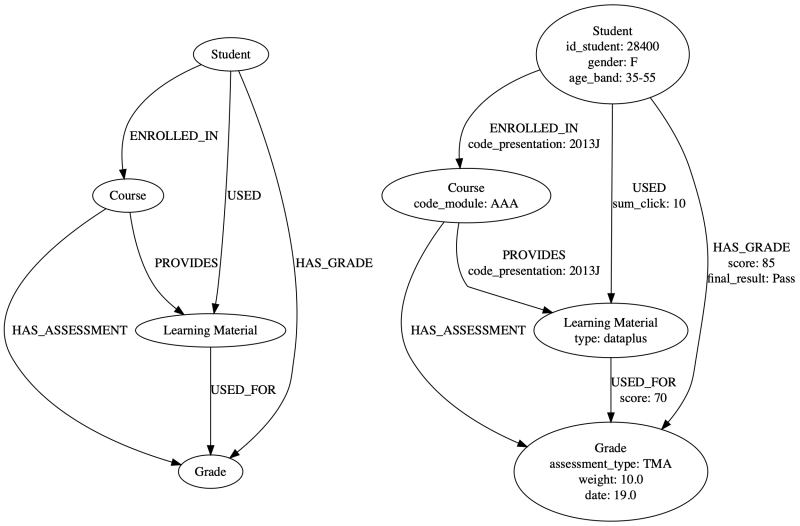

Figure 2: Graph database architecture

Graph databases, like Neo4j, require a specific data modelling approach distinct from relational databases. This involves transforming relational structures into nodes and relationships, optimizing for efficient traversal and query performance [20]. The data model depends on both the data and the queries, guiding the construction of the model accordingly [11].

Entities from relational tables are mapped to nodes in Neo4j, where each node represents a unique entity such as courses, students, or learning materials. Attributes from relational columns are translated into properties of these nodes. For example, a course entity in the relational model, identified by `code_module` and `code_presentation`, becomes a node labeled as `Course` in Neo4j.

Technical primary keys like `id_assessment` were removed to simplify the model, while business primary keys like `code_module` and `code_presentation` were kept to ensure uniqueness for each `Course` node. Foreign key relationships in relational tables are replaced by direct relationships between corresponding nodes in Neo4j. For instance, a `Course` node is connected directly to an `Assessment` node using a relationship like `HAS_ASSESSMENT`. This approach simplifies data retrieval and enhances query performance for tasks like aggregating student grades or tracking learning material usage.

Clicks are now aggregated as attributes on the relationship between students and learning material nodes. This allows for the identification of study materials used by top students across semesters rather than on a specific day. Indexes are strategically employed in Neo4j to optimize query performance by reducing the number of nodes scanned during traversal. Indexes are typically created on properties used in MATCH and WHERE clauses of queries.[13]. For example an index on sum_click in relationships between nodes enhances performance when aggregating student interaction.

Relational Table	`Graph Database Node/Relationship`
Courses Table	Each course in the table, identified uniquely by `code_module` and `code_presentation`, is represented as a node labeled as `Course`. Attribute: `code_module`
StudentInfo Table	Each student, identified by `id_student`, is transformed into a node labeled as `Student`. Attributes: `id_student`, `gender`, `age_band`
VLE Table	Each entry in the Virtual Learning Environment (VLE), identified by `id_site`, becomes a node labeled as `Learning_Material`. Attribute: `type`
StudentAssessment Table	This table primarily establishes relationships rather than becoming nodes itself: - `HAS_ASSESSMENT`: Links courses (`Course`) to assessments (`Grade`). - `HAS_GRADE`: Connects students (`Student`) to their grades (`Grade`) with attributes `score` and `final_result`.
StudentVLE Table	This table establishes relationships: - `USED`: Indicates usage of learning materials (`Learning_Material`) by students (`Student`) with attribute `sum_click`.
Assessments Table	Each assessment, identified by `id_assessment`, is represented as a node labeled as `Grade`. Attributes: `assessment_type`, `weight`, `date`

Table 3: Transformation from Relational Tables to Graph Database Nodes and Relationships

The relational database model was effectively transformed into a graph database model by adopting this approach 3.

3 Performance results of the database queries

The following plots show the response times of the presented queries 1.3 over the day, measured in milliseconds (ms), for the two database systems: the Neo4j graph database, represented by a

blue line plot, and the relational PostgreSQL database, represented by a red line plot. The x-axis represents the time of day (hour), and the y-axis represents the response times in milliseconds.

For the graph database, response times were averaged from 10 queries conducted at specific times: 10 AM, 12 PM, 2 PM, 4 PM, 6 PM, and 8 PM. These averages illustrate typical performance trends throughout the day. Detailed query results in CSV format are provided in the appendix A. The initial query for each hour in the graph database, known as the cold start, exhibited significant delay and was excluded from the averages. Further insights into this issue are discussed in section 4. Due to its local hosting, the relational PostgreSQL database showed consistent response times over ten repeated queries. The average of these results is presented as a constant, in contrast to the varying trends observed in the graph database plots.

The bar plot 3 shows average response times for each query type: graph database (blue) vs. relational database (red). Inserting data averages 42.8 ms for the graph database and 3.2 ms for the relational database. Recommending learning materials takes 11.9 ms on the graph database vs. 73.9 ms on the relational database. Identifying at-risk students averages 20.0 ms on the graph database and 90.9 ms on the relational database. Identifying common courses takes 29.3 ms on the graph database vs. 5.9 ms on the relational database. Querying students in a course takes 9.2 ms on the graph database and 1.1 ms on the relational database.

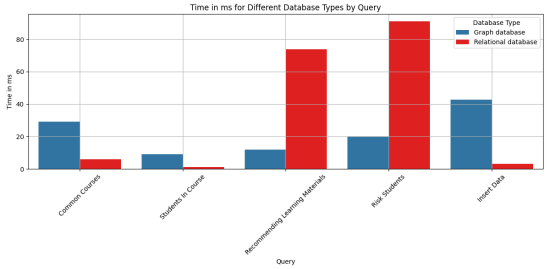

Figure 3: Performance Comparison

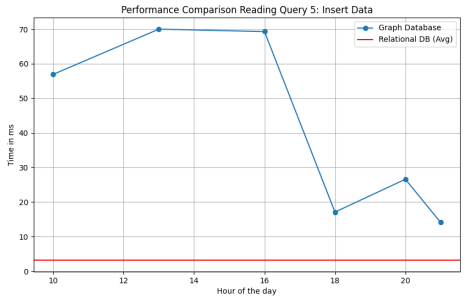

Figure 4: Performance Writing Query 1

The graph 4 shows significant variation in response times throughout the day for the graph database, ranging from 14.1 ms to 70 ms, reflecting insert queries. In contrast, the relational database maintained consistently low response times, averaging 3.2 ms for data insertion.

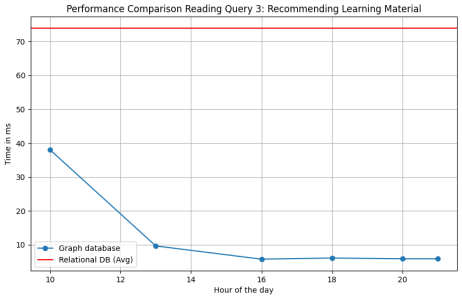

Figure 5: Performance Reading Query 1

For recommending study materials, the graph database showed varying response times, ranging from 5.78 ms to 38 ms, whereas the relational database had a higher average response time of 73.9 ms.

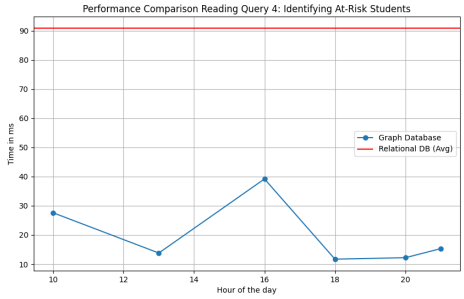

Figure 6: Reading Query 2

For identifying at-risk students, the graph database response times varied between 11.7 ms and 39.2 ms, depending on the time of day. In contrast, the relational database had a higher response time of 90.9 ms.

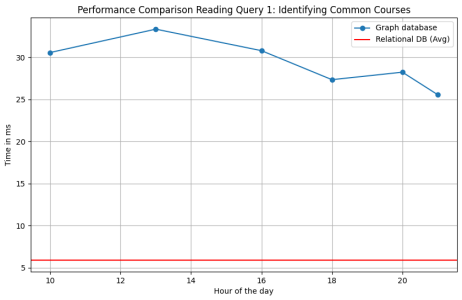

Figure 7: Reading Query 3

Reading Query 3: Matching Study Partners Based on Common Courses For identifying common courses, the graph database response times ranged from 25.6 ms to 33.3 ms throughout the day. In contrast, the relational database consistently maintained a low response time of 5.9 ms.

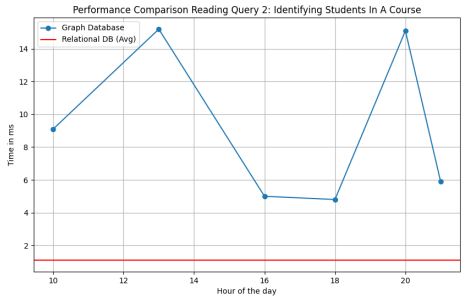

Figure 8: Reading Query 4

Reading Query 4: Querying all students in one course For identifying students in a course, the graph database response times varied from 4.8 ms to 15.2 ms throughout the day. In contrast, the relational database consistently had a low response time of 1.1 ms.

4 Discussion of Results

This section will discuss the former results presented in the section 3.

To provide a balanced evaluation, several limitations must be acknowledged. Tests run with a specific number of entries, which might not reflect the performance with larger datasets. The Neo4j AuraDB Free instance only supports up to 200k nodes and 400k relationships [16]. The online hosting of Neo4j could introduce network latency, unlike the locally hosted PostgreSQL. The

hardware configuration used locally was an Apple MacBook Air with M3 Chip coupled with 16 GB RAM running on macOS Sonoma Version 14.5. Performance may vary throughout the day due to server load and resource availability, common in free versions of software. [16] Differences in query optimization techniques and indexing strategies could affect performance. While efforts were made to optimize queries, there may be more efficient options. Neo4j's initial queries access data from disk before caching, leading to slower first-time query performance. Also running a query for the first time consumes more time as the Cypher Engine has to compile an execution plan. Subsequent queries benefit from faster access speeds due to cached data [26].

By acknowledging these limitations, one can better understand the scope and applicability of the performance comparison results. The table 4 in the appendix summarizes the complexity of the SQL and Cypher queries.

The slower insert performance of the graph database, as observed in response times varying from 14.1ms to 70ms throughout the day, can be attributed to several factors. Graph databases like Neo4j prioritise efficient data analysis over insert operations, focusing on complex relationship modelling. Inserting data into a graph database requires creating and managing relationships between nodes and edges, transaction integrity, and indexing for graph traversal. That can introduce computational overhead compared to simple row insertion in relational databases.[23] Relational databases face increasing execution times for inserts as table sizes grow, unlike graph databases where `CREATE` queries remain consistent.[2, p. 60] When examining the table 1, it becomes evident that every table is relatively small in size, with the exception of the `studentVle`.

In a university context, data updates and inserts are critical not only for ongoing data analysis, but also for managing dynamic scenarios such as new student enrolments, course changes, daily student interactions with learning materials, and updates related to assessments and grades. This requires batch inserts at the start of each semester and periodic updates throughout the semester to ensure that student activity is accurately captured for ongoing analysis and identification of at-risk students.

Graph databases like Neo4j are highly effective for suggesting study materials and identifying at-risk students compared to relational databases. They achieve rapid graph traversals using index-free adjacency, ensuring quick performance regardless of data size. This method directly connects students, courses, and materials without the overhead of traditional SQL queries that rely on complex joins across multiple tables (e.g., `studentInfo`, `studentAssessment`, `courses`, `studentVle`). Index-free adjacency means that connected nodes in a graph refer directly to each other without the need for an index. The advantage is faster and more efficient processing of graph-based queries and operations.[20] For instance, when recommending study materials based on student interactions, graph databases swiftly find relevant connections. In contrast, relational databases require multiple sub-queries, leading to longer execution times, especially for complex queries involving deep relationships (e.g., student engagement metrics) [20, 2]. Graph databases excel because joins or relationships are precomputed, not calculated at query time, resulting in faster processing. Relationships are established at creation, ensuring efficient handling of complex queries without performance penalties [25].

As the size and complexity of queries increase, traditional relational databases may suffer from performance degradation due to intricate data relationships and large database sizes. In contrast, graph databases like Neo4j prioritize relationships, eliminating the need for complex joins and external processing methods like MapReduce [20]. This approach proves advantageous in educational data models where students are linked to courses, grades, assessments, and learning materials via various relationships. Graph databases excel in efficiently running queries that involve path

traversals, such as aggregating statistics based on student interactions with learning materials or identifying common courses among students [20].

Finally, the enhanced performance of the relational database in the reading queries illustrated in figures 7 and 8 is due to the limited number of relationships that the query has to search. To query all students in one course, it is only necessary to iterate over one table. Furthermore, to identify matching study partners, only a single `JOIN` operation is required. With a low number of `JOIN`s, the execution time of search queries in relational databases improves significantly.[25] In contrast, in Neo4j, queries involve traversing relationships (such as `ENROLLED_IN`) between nodes (such as Course and Student). This traversal can involve scanning large parts of the graph to find nodes that match the specified relationship pattern. While graph databases excel at traversing complex relationships, relational databases are often optimised for direct access and join operations through effective indexing strategies.

5 Theoretical Outlook

To consider the dependency on Neo4j, it is clear that several graph query languages exist, such as SPARQL. Despite Cypher being developed by Neo4j, it is the most widely used and therefore a standard and can be used with other graph database products [21, p. 25]. Neo4j offers drivers and APIs for various programming languages (e.g., Python, Java, JavaScript) to facilitate data access [15]. However, vendor lock-in can occur as Cypher and property graph database specifics are less widespread than SQL. Tools like the Neo4j to OrientDB Importer can help switch deployment systems [18]. Besides that, Neo4j offers a wide selection of techniques to faciliate data import and guarantee optimal performance, which illustrates a crucial task when migrating between two different systems. One method is to use the batch-import project where data can be imported from CSV files to nodes and relationships. The batch-import project then generates the Neo4j data set and an index structure, which is effective as it builds the index from the ground up, ensuring efficient data organization and retrieval. Furthermore, Neo4j provides a `LOAD_CSV` technique, which facilitates the importation of data from the graph database into CSV files. This method is particularly adept at processing large amounts of data in a timely manner [8].

Neo4j utilises a master-slave cluster arrangement where each machine stores a complete replica of the graph. This ensures that if one node fails, other instances are ready to process requests without the need for human intervention. This is particularly important in the presented use case where continuous access to student data, learning materials and analytics is required to support learning success. Writes are continuously replicated from the master to the slaves, ensuring all nodes are nearly synchronized, with minimal delay of milliseconds, which is important for the accuracy of the student data analyses. [21, p. 164]

Neo4j's architecture supports seamless data migrations while maintaining ACID guarantees, essential for handling large-scale data transitions with minimal disruption. This capability is particularly beneficial for educational platforms adapting to increasing data demands while maintaining real-time responsiveness and operational stability [7].

In educational environments, managing increasing volumes of student data, learning interactions, and real-time analytics requires scalable data management systems. For Neo4j, performance times are nearly independent of the size of the graph, but the query being executed. Neo4j provides a robust capacity, with the capability to support graphs containing tens of billions of nodes, relationships, and properties. To achieve higher troughput or handle data growth on can scale horizontally using Neo4js advanced Autonomous Clustering. This offers fault tolerance for primary copies (zero data

loss with synchronous replication between each copy) and unlimited scale-out for read workloads using secondary copies (using asynchronous replication).[22]. This is a good solution when the use case focus on reads over writes, as its the case for the university environment. If the capacity limit of a cluster is reached sharding using Neo4j Fabric can be a solution. Scaling becomes easy as a large graph is then stored in a small number of independent graphs, enabling unlimited horizontal scale-out.[21, pp. 168, 169]. Should the popularity of the recommendation system increase and the number of read interactions rise, scaling will be necessary.

According to the CAP (consistency, availability, partition tolerance) theorem, Neo4j prioritizes consistency and partition tolerance over availability. During network partitions, Neo4j maintains data consistency but may temporarily reduce availability to ensure data integrity and operational reliability [5]. Accurate and consistent student data is crucial for making informed decisions regarding risk students. It is therefore vital for Neo4j to guarantee consistency, even when multiple users are accessing and updating data simultaneously.

In the context of ongoing educational analysis, it is crucial to have the ability to handle changes to the graph. The addition of new nodes and relationships does not typically have a significant impact on existing queries. However, altering existing relationships or node structures requires testing to ensure everything functions correctly. These changes are generally simpler compared to other database systems. As the application grows in popularity, new functionalities could be added, such as personalized recommendations based on student interests, learning styles, and past interactions, or feedback options for students on recommended learning materials.[21, p. 58]

To manage schema changes, Neo4j-Migrations provides a comprehensive set of tools that simplify the process of tracking, managing and applying schema changes to Neo4j databases. These tools are inspired by FlywayDB and provide a uniform method for applications, command-line tools and build tools. Neo4j-Migrations can be employed against Neo4j 3.5, all Neo4j 4 versions from 4.1 up to 4.4 and Neo4j 5, including all current Neo4j-Aura versions.[24]

6 Conclusion and Recommendations

The analysis indicates that Neo4j is well-suited for the management of complex relationships and the conduct of efficient queries in educational settings. It is particularly efficient at tasks such as real-time analytics and personalised learning. However, the results show that it is less efficient than traditional databases such as PostgreSQL in data insertion tasks.

In order to optimize the utilisation of Neo4j in an educational context, one can recommend that data inserts are conducted via batch processing during periods of low activity, in order to enhance performance during periods of high activity. Another solution would be using tools such as `LOAD_CSV` to effective bulk import the data from CSV-files into the database described in [14]. Furthermore, upgrading to Neo4j's paid version may more features and scalability options, which are necessary for managing increasing amounts of data. In summary, Neo4j illustrates a fitting choice for the analysis of university data due to its ability to handle complex connections. By implementing improved data insertion methods and considering payed upgrades, Neo4j offers a an optimized tool for educational needs.

Link to repository

References

[1] Balqis Albreiki, Tetiana Habuza, and Nazar Zaki. "Extracting Topological Features to Identify At-Risk Students Using Machine Learning and Graph Convolutional Network Models". In: *International Journal of Educational Technology in Higher Education* 20.1 (2023), p. 23.

[2] Yaowen Chen. *Comparison of Graph Databases and Relational Databases When Handling Large-Scale Social Data*. Master's thesis. Available at `https://harvest.usask.ca/server/api/core/bitstreams/a5941aea-18a9-46ed-a8c6-21237a260969/content`. Saskatoon, Saskatchewan Canada, Sept. 2016.

[3] C. J. Date and Ronald Fagin. "Simple conditions for guaranteeing higher normal forms in relational databases". In: *ACM Trans. Database Syst.* 17.3 (Sept. 1992), pp. 465–476. ISSN: 0362-5915. DOI: 10.1145/132271.132274. URL: https://doi.org/10.1145/132271.132274.

[4] Ali Davoudian, Liu Chen, and Mengchi Liu. "A Survey on NoSQL Stores". In: *ACM Comput. Surv.* 1.1 (Jan. 2017), Article 1. DOI: 10.1145/3158661. URL: https://doi.org/10.1145/3158661.

[5] GeeksforGeeks. *The CAP Theorem in DBMS*. `https://www.geeksforgeeks.org/the-cap-theorem-in-dbms/`. Accessed: 2024-06-25.

[6] Michael Hunger, Ryan Boyd, and William Lyon. "Conceptual and Database Modelling of Graph Databases". In: *Proceedings of the 20th International Database Engineering & Applications Symposium (IDEAS '16)*. 2017.

[7] Mit Jain, Ashish Khanchandani, and Cajetan Rodrigues. "Performance Comparison of Graph Database and Relational Database". In: San Jose State University, Computer Science Department. San Jose, USA, 2023.

[8] Juan Pablo Albuja. *Exploring Data Migration Techniques with Neo4j*. `https://neo4j.com/blog/exploring-data-migration-techniques-neo4j/`. Accessed: 2024-07-02.

[9] William Kent. "A simple guide to five normal forms in relational database theory". In: *Communications of the ACM* 26.2 (1983), pp. 120–125.

[10] Donald E. Knuth. *The TEX Book*. Addison-Wesley Professional, 1986.

[11] maxdemarzi. *Modeling Airline Flights in Neo4j*. `https://maxdemarzi.com/2015/08/26/modeling-airline-flights-in-neo4j`. Accessed: 2024-06-29. Aug. 2015.

[12] Steve Ataky Tsham Mpinda et al. "Evaluation of Graph Databases Performance Through Indexing Techniques". In: *International Journal of Artificial Intelligence & Applications (IJAIA)* 6.5 (Sept. 2015), pp. 1–15.

[13] Steve Ataky Tsham Mpinda et al. "Evaluation of graph databases performance through indexing techniques". In: *International Journal of Artificial Intelligence & Applications (IJAIA)* 6.5 (Sept. 2015).

[14] Michael Hunger Mark Needham. *Effective Bulk Data Import into Neo4j*. Neo4j. URL: `https://neo4j.com/blog/bulk-data-import-neo4j-3-0/`.

[15] Neo4j. *Connect to Neo4j*. `https://neo4j.com/docs/getting-started/languages-guides/`.

[16] Neo4j. *Neo4j AuraDB Pricing*. Accessed: 2024-06-27. 2024. URL: `https://neo4j.com/pricing/`.

[17] J. Novak. "Clarify with Concept Maps". In: *The Science Teacher* 58.7 (1991), p. 44.

[18] OrientDB. *OrientDB-Neo4j Importer*. `https://orientdb.com/docs/2.2.x/OrientDB-Neo4j-Importer.html`.

[19] J. Bryan Osborne and Andrew S. I. D. Lang. "Predictive Identification of At-Risk Students: Using Learning Management System Data". In: *Journal of Postsecondary Student Success* 2.4 (2023).

[20] Jaroslav Pokorný. *The Definitive Guide to Graph Databases for the RDBMS Developer*. neo4j.com. 2021.

[21] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases, 2nd Edition*. O'Reilly Media, Inc., June 2015. ISBN: 9781491930892.

[22] *Scaling Strategies with Neo4j*. neo4j.com. 2022. URL: `https://neo4j.com/whitepapers/scaling-strategies-with-neo4j/`.

[23] Amazon Web Services. *The Difference Between Graph and Relational Database*. Amazon Web Services. URL: `https://aws.amazon.com/compare/the-difference-between-graph-and-relational-database/`.

[24] Michael Simons. *Neo4j-Migrations*. `https://github.com/michael-simons/neo4j-migrations`.

[25] Liana Stanescu. "A Comparison between a Relational and a Graph Database in the Context of a Recommendation System". In: *Position and Communication Papers of the 16th Conference on Computer Science and Intelligence Systems*. Vol. 26. ACSIS. Email: stanescu@software.ucv.ro. University of Craiova, Faculty of Automation, Computers and Electronics. Craiova, Romania: IEEE, 2021, pp. 133–139. DOI: `10.15439/2021F33`.

[26] Kees Vegter. *Cypher Query Optimisations*. Neo4j Developer Blog. Nov. 2018. URL: `https://medium.com/neo4j/cypher-query-optimisations-fe0539ce2e5c`.

[27] Qiang Wang. "PostgreSQL database performance optimization". In: (2011).

A Appendix

Writing Query 1: Insert data into the databases

```sql
-- Insert

   -- Insert into courses
INSERT INTO courses (code_module, code_presentation) VALUES
      ('xxx', '2024J');

-- Insert into vle
INSERT INTO vle (id_site, code_module, code_presentation,
      activity_type) VALUES (100000, 'xxx', '2024J', 'forum'
      );

-- Insert into assessments
INSERT INTO assessments (id_assessment, code_module,
      code_presentation, assessment_type, date, weight)
      VALUES (100000, 'xxx', '2024J', 'TMA', 100, 20);

-- Insert into studentInfo
INSERT INTO studentInfo (id_student, code_module,
      code_presentation, gender, final_result) VALUES
      (00000, 'xxx', '2024J', 'M', 'Pass');

-- Insert into studentAssessment
INSERT INTO studentAssessment (id_assessment, id_student,
      score) VALUES (100000, 00000, 85.0);

-- Insert into studentVle
INSERT INTO studentVle (code_module, code_presentation,
      id_student, id_site, sum_click) VALUES ('xxx', '2024J'
      , 00000, 100000,  10);
```

Listing 1: SQL Query

```cypher
-- insert
UNWIND [
  {id_student: "0000", gender: "M", age_band: "0-35",
        code_module: "xxx", activity_type: "test",
        code_presentation: "2024J", sum_click: 100,
        assessment_type: "test", weight: 10, date: "
        test_date", score: 85, final_result: "Pass"}
] AS row
MERGE (s:Student {id_student: row.id_student})
ON CREATE SET s.gender = row.gender, s.age_band = row.
      age_band
MERGE (c:Course {code_module: row.code_module})
MERGE (learning:Learning_Material {type: row.activity_type})
MERGE (s)-[:ENROLLED_IN  { code_presentation: row.
      code_presentation }]->(c)
MERGE (s)-[:USED  { sum_click : row.sum_click }]->(learning)
MERGE (c)-[:PROVIDES  { code_presentation  : row.
      code_presentation }]->(learning)
MERGE (g:Grade {assessment_type: row.assessment_type, weight
      : row.weight, date: row.date})
MERGE (s)-[:HAS_GRADE  { score  : row.score, final_result :
      row.final_result }]->(g)
MERGE (c)-[:HAS_ASSESSMENT]->(g)
MERGE (learning)-[:USED_FOR {score: row.score}]->(g)
```

Listing 2: Cypher Query

Reading Query 1: Recommendation of Learning Materials Based on Top Students

```sql
WITH QualifiedStudents AS (
    SELECT si.id_student
    FROM studentInfo si
    JOIN studentAssessment sa ON si.id_student = sa.
          id_student
    JOIN courses c ON si.code_module = c.code_module AND si.
          code_presentation = c.code_presentation
    WHERE c.code_module = 'AAA' AND LEFT(si.
          code_presentation, 4)::INT > 2013 AND sa.score >
          50.0
),
MostUsedMaterial AS (
    SELECT sv.id_student, v.activity_type, SUM(sv.sum_click)
          AS total_clicks
    FROM studentVle sv
    JOIN vle v ON sv.id_site = v.id_site
    WHERE sv.id_student IN (SELECT id_student FROM
          QualifiedStudents)
    GROUP BY sv.id_student, v.activity_type
    ORDER BY total_clicks DESC
)
SELECT activity_type, sum(total_clicks) AS clicks_total
FROM MostUsedMaterial
GROUP BY activity_type
ORDER BY clicks_total DESC;
```

Listing 3: SQL Query

```cypher
// First MATCH clause to filter relevant courses and
      students
MATCH (c:Course {code_module: "AAA"})
WITH c

// Second MATCH clause to filter relevant ENROLLED_IN
      relationships
MATCH (s:Student)-[e:ENROLLED_IN]->(c)
WHERE toInteger(substring(e.code_presentation, 0, 4)) > 2013
WITH s, c

// Third MATCH clause to filter relevant HAS_GRADE
      relationships
MATCH (s)-[r:HAS_GRADE]->(g:Grade)-[:HAS_ASSESSMENT]-(c)
WHERE r.score > 50.0
WITH COLLECT(DISTINCT s) AS students

// Unwind to process the collected students further
UNWIND students AS s

// Fourth MATCH clause to process USED relationships and
      aggregate clicks
MATCH (s)-[u:USED]->(m:Learning_Material)
RETURN m.type as materialType, sum(u.sum_click) as
      total_clicks
ORDER BY total_clicks DESC;
```

Listing 4: Cypher Query

Reading Query 2: Identifying At-Risk Students through Engagement Analysis

```
WITH AverageClicks AS (
SELECT
    SUM(sv.sum_click) / ( SELECT COUNT(DISTINCT si.
        id_student)
                        FROM studentInfo si
                        WHERE si.code_module = 'AAA'
                        AND si.code_presentation = '2013J'
        ) AS average_clicks
FROM
    studentVle sv
JOIN
    vle ON sv.id_site = vle.id_site
WHERE
    sv.code_module = 'AAA'
    AND sv.code_presentation = '2013J'
),

TotalClicks AS (
    SELECT
        sv.id_student,
        SUM(sv.sum_click) AS total_clicks
    FROM
        studentVle sv
    WHERE
        sv.code_module = 'AAA'
        AND sv.code_presentation = '2013J'
    GROUP BY
        sv.id_student
)

select
    tc.id_student AS risk_student,
    tc.total_clicks AS clicks,
    ac.average_clicks
FROM
    TotalClicks tc, AverageClicks ac
WHERE
    tc.total_clicks < ac.average_clicks;
```

Listing 5: SQL Query

```
MATCH (c:Course {code_module: "AAA"})<-[:ENROLLED_IN {
      code_presentation: "2013J"}]-(s:Student)-[click:USED
      ]->(lm:Learning_Material)
WITH count(distinct s.id_student) as studentCount, sum(click
      .sum_click) as totalClicks
WITH studentCount, totalClicks, totalClicks / studentCount
      as averageClicks

MATCH (c:Course {code_module: "AAA"})<-[:ENROLLED_IN {
      code_presentation: "2013J"}]-(s:Student)-[click:USED
      ]->(lm:Learning_Material)
WITH s, sum(click.sum_click) as clicks, averageClicks
WHERE clicks < averageClicks
RETURN s.id_student as id_student, clicks, averageClicks
```

Listing 6: Cypher Query

Reading Query 3: Connecting Students: Matching Study Partners Based on Common Courses

```
    SELECT
    s2.id_student,
    COUNT(*) AS common_courses,
    s2.code_module
FROM
    studentInfo s1
JOIN studentInfo s2 ON
    s1.code_module = s2.code_module
    AND s1.code_presentation = s2.code_presentation
    AND s1.id_student != s2.id_student
WHERE
    s1.id_student = 38053
GROUP BY
    s2.id_student,
    s2.code_module
ORDER BY
    common_courses DESC,
    s2.id_student;
```

Listing 7: SQL Query

```
MATCH (s1:Student {id_student: "38053"})-[e1:ENROLLED_IN]->(
      c:Course)<-[e2:ENROLLED_IN]-(s2:Student)
WHERE e1.code_presentation = e2.code_presentation
WITH s1, s2, c, count(c) as common_courses
ORDER BY common_courses DESC
RETURN s2.id_student, common_courses, c.code_module
```

Listing 8: Cypher Query

Reading Query 4: Querying all students in one course

```
SELECT
    id_student
FROM
    studentInfo s
WHERE
    s.code_module = 'AAA'
    AND s.code_presentation = '2013J';
```

Listing 9: SQL Query

```
MATCH (c:Course {code_module: "AAA"})<-[:ENROLLED_IN
      {code_presentation: "2013J"}]-(s:Student)
RETURN c.code_module, collect(s.id_student) as
      students
```

Listing 10: Cypher Query

Table 4: Summary of Queries and Their Complexity

Query	SQL Complexity	Cypher Complexity
Writing Query 1	Moderate complexity due to `INSERT` operations involving multiple tables and relationships.	Complex due to `MERGE` operations for graph traversal and node/relationship creation.
Reading Query 1	Complex due to `JOIN`, subquery for qualified students, and `GROUP BY` aggregation and `ORDER BY` statement.	Moderate complexity, involves `MATCH` for node matching and `WITH` for aggregation. Also `COLLECT(DISTINCT)`.
Reading Query 2	Involves complex `JOIN` statements across multiple tables and `SELECT COUNT (DISTINTC), WHERE`	Moderate complexity, involves `MATCH` for node matching and `WHERE` for condition filtering. The query identifies at-risk students by aggregating engagement metrics.
Reading Query 3	Complex due to self-`JOIN`, `GROUP BY` operations, and `COUNT` aggregation.	Moderate complexity, involves multiple `MATCH` clauses for node matching and `RETURN` for result specification.
Reading Query 4	Simple, straightforward `SELECT` query with `WHERE` condition.	Moderate, the query involves traversing relationships (`ENROLLED_IN`) between nodes (`Course` and `Student`).

Table 5: Performance of Insert Queries on Relational and Graph Databases

Query	**Database Type**	**Hour of the day**	**Time in ms**	**Run ID**
Insert	Relational Database	10	4	1
Insert	Relational Database	10	3	2
Insert	Relational Database	10	3	3
Insert	Relational Database	10	4	4
Insert	Relational Database	10	3	5
Insert	Relational Database	10	3	6
Insert	Relational Database	10	3	7
Insert	Relational Database	10	3	8
Insert	Relational Database	10	3	9
Insert	Relational Database	10	3	10
Insert	Graph Database	10	231	1
Insert	Graph Database	10	224	2
Insert	Graph Database	10	35	3
Insert	Graph Database	10	33	4
Insert	Graph Database	10	28	5
Insert	Graph Database	10	31	6
Insert	Graph Database	10	9	7
Insert	Graph Database	10	10	8
Insert	Graph Database	10	12	9

Query	Database Type	Hour of the day	Time in ms	Run ID
Insert	Graph Database	10	9	10
Insert	Graph Database	10	8	11
Insert	Graph Database	10	53	12
Insert	Graph Database	13	454	1
Insert	Graph Database	13	48	2
Insert	Graph Database	13	47	3
Insert	Graph Database	13	40	4
Insert	Graph Database	13	37	5
Insert	Graph Database	13	34	6
Insert	Graph Database	13	10	7
Insert	Graph Database	13	11	8
Insert	Graph Database	13	10	9
Insert	Graph Database	13	9	10
Insert	Graph Database	16	438	1
Insert	Graph Database	16	33	2
Insert	Graph Database	16	36	3
Insert	Graph Database	16	44	4
Insert	Graph Database	16	35	5
Insert	Graph Database	16	39	6
Insert	Graph Database	16	35	7
Insert	Graph Database	16	14	8
Insert	Graph Database	16	9	9
Insert	Graph Database	16	10	10
Insert	Graph Database	18	66	1
Insert	Graph Database	18	13	2
Insert	Graph Database	18	16	3
Insert	Graph Database	18	11	4
Insert	Graph Database	18	11	5
Insert	Graph Database	18	11	6
Insert	Graph Database	18	11	7
Insert	Graph Database	18	12	8
Insert	Graph Database	18	11	9
Insert	Graph Database	18	9	10
Insert	Graph Database	20	38	1
Insert	Graph Database	20	37	2
Insert	Graph Database	20	35	3
Insert	Graph Database	20	30	4
Insert	Graph Database	20	30	5
Insert	Graph Database	20	12	6
Insert	Graph Database	20	10	7
Insert	Graph Database	20	10	8
Insert	Graph Database	20	55	9
Insert	Graph Database	20	9	10
Insert	Graph Database	21	34	1
Insert	Graph Database	21	14	2

Query	Database Type	Hour of the day	Time in ms	Run ID
Insert	Graph Database	21	19	3
Insert	Graph Database	21	10	4
Insert	Graph Database	21	14	5
Insert	Graph Database	21	10	6
Insert	Graph Database	21	10	7
Insert	Graph Database	21	11	8
Insert	Graph Database	21	11	9
Insert	Graph Database	21	8	10

Table 6: Performance of Recommendations Queries on Relational and Graph Databases

Query	Database Type	Hour of the day	Time in ms	Run ID
Recommendations	Relational Database	10	170	1
Recommendations	Relational Database	10	76	2
Recommendations	Relational Database	10	65	3
Recommendations	Relational Database	10	61	4
Recommendations	Relational Database	10	62	5
Recommendations	Relational Database	10	66	6
Recommendations	Relational Database	10	61	7
Recommendations	Relational Database	10	52	8
Recommendations	Relational Database	10	62	9
Recommendations	Relational Database	10	64	10
Recommendations	Graph database	10	27	1
Recommendations	Graph database	10	6	2
Recommendations	Graph database	10	6	3
Recommendations	Graph database	10	7	4
Recommendations	Graph database	10	9	5
Recommendations	Graph database	10	7	6
Recommendations	Graph database	10	7	7
Recommendations	Graph database	10	6	8
Recommendations	Graph database	10	285	9
Recommendations	Graph database	10	9	10
Recommendations	Graph database	13	53	1
Recommendations	Graph database	13	6	2
Recommendations	Graph database	13	6	3
Recommendations	Graph database	13	7	4
Recommendations	Graph database	13	6	5
Recommendations	Graph database	13	6	6
Recommendations	Graph database	13	6	7
Recommendations	Graph database	13	6	8
Recommendations	Graph database	13	6	9
Recommendations	Graph database	13	38	10
Recommendations	Graph database	16	480	1

Query	Database Type	Hour of the day	Time in ms	Run ID
Recommendations	Graph database	16	8	2
Recommendations	Graph database	16	6	3
Recommendations	Graph database	16	4	4
Recommendations	Graph database	16	6	5
Recommendations	Graph database	16	8	6
Recommendations	Graph database	16	5	7
Recommendations	Graph database	16	5	8
Recommendations	Graph database	16	5	9
Recommendations	Graph database	16	5	10
Recommendations	Graph database	18	51	1
Recommendations	Graph database	18	9	2
Recommendations	Graph database	18	6	3
Recommendations	Graph database	18	6	4
Recommendations	Graph database	18	6	5
Recommendations	Graph database	18	4	6
Recommendations	Graph database	18	6	7
Recommendations	Graph database	18	6	8
Recommendations	Graph database	18	6	9
Recommendations	Graph database	18	6	10
Recommendations	Graph database	20	50	1
Recommendations	Graph database	20	6	2
Recommendations	Graph database	20	6	3
Recommendations	Graph database	20	6	4
Recommendations	Graph database	20	6	5
Recommendations	Graph database	20	6	6
Recommendations	Graph database	20	5	7
Recommendations	Graph database	20	6	8
Recommendations	Graph database	20	5	9
Recommendations	Graph database	20	7	10
Recommendations	Graph database	21	94	1
Recommendations	Graph database	21	9	2
Recommendations	Graph database	21	6	3
Recommendations	Graph database	21	5	4
Recommendations	Graph database	21	6	5
Recommendations	Graph database	21	6	6
Recommendations	Graph database	21	5	7
Recommendations	Graph database	21	5	8
Recommendations	Graph database	21	6	9
Recommendations	Graph database	21	5	10

Table 7: Performance of Risk Students Queries on Relational and Graph Databases

Query	Database Type	Hour of the day	Time in ms	Run ID
Risk students	Relational Database	13	74	1
Risk students	Relational Database	13	95	2
Risk students	Relational Database	13	92	3
Risk students	Relational Database	13	94	4
Risk students	Relational Database	13	90	5
Risk students	Relational Database	13	90	6
Risk students	Relational Database	13	94	7
Risk students	Relational Database	13	96	8
Risk students	Relational Database	13	91	9
Risk students	Relational Database	13	93	10
Risk students	Graph Database	10	195	1
Risk students	Graph Database	10	9	2
Risk students	Graph Database	10	12	3
Risk students	Graph Database	10	12	4
Risk students	Graph Database	10	9	5
Risk students	Graph Database	10	8	6
Risk students	Graph Database	10	9	7
Risk students	Graph Database	10	7	8
Risk students	Graph Database	10	8	9
Risk students	Graph Database	10	7	10
Risk students	Graph Database	13	66	1
Risk students	Graph Database	13	9	2
Risk students	Graph Database	13	7	3
Risk students	Graph Database	13	7	4
Risk students	Graph Database	13	7	5
Risk students	Graph Database	13	12	6
Risk students	Graph Database	13	8	7
Risk students	Graph Database	13	7	8
Risk students	Graph Database	13	8	9
Risk students	Graph Database	13	7	10
Risk students	Graph Database	16	322	1
Risk students	Graph Database	16	8	2
Risk students	Graph Database	16	7	3
Risk students	Graph Database	16	11	4
Risk students	Graph Database	16	7	5
Risk students	Graph Database	16	9	6
Risk students	Graph Database	16	7	7
Risk students	Graph Database	16	7	8
Risk students	Graph Database	16	7	9
Risk students	Graph Database	16	7	10
Risk students	Graph Database	18	53	1
Risk students	Graph Database	18	7	2

Query	Database Type	Hour of the day	Time in ms	Run ID
Risk students	Graph Database	18	6	3
Risk students	Graph Database	18	8	4
Risk students	Graph Database	18	6	5
Risk students	Graph Database	18	8	6
Risk students	Graph Database	18	7	7
Risk students	Graph Database	18	7	8
Risk students	Graph Database	18	7	9
Risk students	Graph Database	18	8	10
Risk students	Graph Database	20	58	1
Risk students	Graph Database	20	9	2
Risk students	Graph Database	20	6	3
Risk students	Graph Database	20	7	4
Risk students	Graph Database	20	7	5
Risk students	Graph Database	20	6	6
Risk students	Graph Database	20	6	7
Risk students	Graph Database	20	8	8
Risk students	Graph Database	20	7	9
Risk students	Graph Database	20	8	10
Risk students	Graph Database	21	80	1
Risk students	Graph Database	21	8	2
Risk students	Graph Database	21	9	3
Risk students	Graph Database	21	6	4
Risk students	Graph Database	21	8	5
Risk students	Graph Database	21	9	6
Risk students	Graph Database	21	10	7
Risk students	Graph Database	21	8	8
Risk students	Graph Database	21	7	9
Risk students	Graph Database	21	8	10

Table 8: Performance of Common Courses Queries on Relational and Graph Databases

Query	Database Type	Hour of the day	Time in ms	Run ID
Common courses	Relational Database	13	5	1
Common courses	Relational Database	13	6	2
Common courses	Relational Database	13	6	3
Common courses	Relational Database	13	6	4
Common courses	Relational Database	13	7	5
Common courses	Relational Database	13	6	6
Common courses	Relational Database	13	8	7
Common courses	Relational Database	13	4	8
Common courses	Relational Database	13	6	9
Common courses	Relational Database	13	5	10
Common courses	Relational Database	10	9	1

Query	Database Type	Hour of the day	Time in ms	Run ID
Common courses	Relational Database	10	6	2
Common courses	Relational Database	10	6	3
Common courses	Relational Database	10	4	4
Common courses	Relational Database	10	6	5
Common courses	Relational Database	10	7	6
Common courses	Relational Database	10	5	7
Common courses	Relational Database	10	4	8
Common courses	Relational Database	10	6	9
Common courses	Relational Database	10	6	10
Common courses	Graph Database	10	232	1
Common courses	Graph Database	10	40	2
Common courses	Graph Database	10	25	3
Common courses	Graph Database	10	37	4
Common courses	Graph Database	10	26	5
Common courses	Graph Database	10	28	6
Common courses	Graph Database	10	43	7
Common courses	Graph Database	10	25	8
Common courses	Graph Database	10	26	9
Common courses	Graph Database	10	25	10
Common courses	Graph Database	13	559	1
Common courses	Graph Database	13	52	2
Common courses	Graph Database	13	25	3
Common courses	Graph Database	13	28	4
Common courses	Graph Database	13	36	5
Common courses	Graph Database	13	40	6
Common courses	Graph Database	13	31	7
Common courses	Graph Database	13	39	8
Common courses	Graph Database	13	23	9
Common courses	Graph Database	13	26	10
Common courses	Graph Database	16	155	1
Common courses	Graph Database	16	32	2
Common courses	Graph Database	16	24	3
Common courses	Graph Database	16	40	4
Common courses	Graph Database	16	40	5
Common courses	Graph Database	16	33	6
Common courses	Graph Database	16	23	7
Common courses	Graph Database	16	37	8
Common courses	Graph Database	16	24	9
Common courses	Graph Database	16	24	10
Common courses	Graph Database	18	61	1
Common courses	Graph Database	18	28	2
Common courses	Graph Database	18	33	3
Common courses	Graph Database	18	25	4
Common courses	Graph Database	18	36	5
Common courses	Graph Database	18	24	6

Query	Database Type	Hour of the day	Time in ms	Run ID
Common courses	Graph Database	18	25	7
Common courses	Graph Database	18	26	8
Common courses	Graph Database	18	25	9
Common courses	Graph Database	18	24	10
Common courses	Graph Database	20	256	1
Common courses	Graph Database	20	31	2
Common courses	Graph Database	20	25	3
Common courses	Graph Database	20	26	4
Common courses	Graph Database	20	39	5
Common courses	Graph Database	20	24	6
Common courses	Graph Database	20	26	7
Common courses	Graph Database	20	24	8
Common courses	Graph Database	20	24	9
Common courses	Graph Database	20	35	10
Common courses	Graph Database	21	294	1
Common courses	Graph Database	21	28	2
Common courses	Graph Database	21	25	3
Common courses	Graph Database	21	25	4
Common courses	Graph Database	21	24	5
Common courses	Graph Database	21	27	6
Common courses	Graph Database	21	27	7
Common courses	Graph Database	21	26	8
Common courses	Graph Database	21	24	9
Common courses	Graph Database	21	24	10

Table 9: Performance of Students in a Course Queries on Relational and Graph Databases

Query	Database Type	Hour of the day	Time in ms	Run ID
Students in a course	Relational Database	10	1	1
Students in a course	Relational Database	10	1	2
Students in a course	Relational Database	10	1	3
Students in a course	Relational Database	10	1	4
Students in a course	Relational Database	10	1	5
Students in a course	Relational Database	10	1	6
Students in a course	Relational Database	10	1	7
Students in a course	Relational Database	10	1	8
Students in a course	Relational Database	10	2	9
Students in a course	Relational Database	10	1	10
Students in a course	Graph Database	10	72	1
Students in a course	Graph Database	10	2	2
Students in a course	Graph Database	10	2	3
Students in a course	Graph Database	10	2	4
Students in a course	Graph Database	10	3	5

Query	Database Type	Hour of the day	Time in ms	Run ID
Students in a course	Graph Database	10	2	6
Students in a course	Graph Database	10	2	7
Students in a course	Graph Database	10	1	8
Students in a course	Graph Database	10	3	9
Students in a course	Graph Database	10	2	10
Students in a course	Graph Database	13	134	1
Students in a course	Graph Database	13	2	2
Students in a course	Graph Database	13	1	3
Students in a course	Graph Database	13	2	4
Students in a course	Graph Database	13	2	5
Students in a course	Graph Database	13	3	6
Students in a course	Graph Database	13	2	7
Students in a course	Graph Database	13	2	8
Students in a course	Graph Database	13	2	9
Students in a course	Graph Database	13	2	10
Students in a course	Graph Database	16	31	1
Students in a course	Graph Database	16	3	2
Students in a course	Graph Database	16	2	3
Students in a course	Graph Database	16	1	4
Students in a course	Graph Database	16	2	5
Students in a course	Graph Database	16	2	6
Students in a course	Graph Database	16	2	7
Students in a course	Graph Database	16	2	8
Students in a course	Graph Database	16	3	9
Students in a course	Graph Database	16	2	10
Students in a course	Graph Database	18	29	1
Students in a course	Graph Database	18	3	2
Students in a course	Graph Database	18	2	3
Students in a course	Graph Database	18	1	4
Students in a course	Graph Database	18	2	5
Students in a course	Graph Database	18	3	6
Students in a course	Graph Database	18	2	7
Students in a course	Graph Database	18	3	8
Students in a course	Graph Database	18	2	9
Students in a course	Graph Database	18	1	10
Students in a course	Graph Database	20	129	1
Students in a course	Graph Database	20	5	2
Students in a course	Graph Database	20	2	3
Students in a course	Graph Database	20	1	4
Students in a course	Graph Database	20	1	5
Students in a course	Graph Database	20	3	6
Students in a course	Graph Database	20	4	7
Students in a course	Graph Database	20	2	8
Students in a course	Graph Database	20	2	9
Students in a course	Graph Database	20	2	10

Query	Database Type	Hour of the day	Time in ms	Run ID
Students in a course	Graph Database	21	39	1
Students in a course	Graph Database	21	2	2
Students in a course	Graph Database	21	1	3
Students in a course	Graph Database	21	3	4
Students in a course	Graph Database	21	2	5
Students in a course	Graph Database	21	2	6
Students in a course	Graph Database	21	3	7
Students in a course	Graph Database	21	3	8
Students in a course	Graph Database	21	2	9
Students in a course	Graph Database	21	2	10

activity_type	clicks_total
oucontent	178824
forumng	106556
homepage	85938
subpage	21748
url	8448
resource	5268
dataplus	1103
oucollaborate	178
glossary	170

Table 10: Query 1: Results Recommendations, 9 rows total

risk_student	clicks	average_clicks
2197016	1455	1774
318933	1281	1774
2523736	470	1774
205719	333	1774
...	...	...

Table 11: Query 2: Results At-Risk Students, 185 rows total

id_student	common_courses	code_module
28400	1	AAA
31604	1	AAA
32885	1	AAA
45462	1	AAA
...	...	...

Table 12: Query 3: Results Common Courses, 294 rows total

id_student
28400
31604
32885
38053
...

Table 13: Query 4: Results Students in course, 295 rows total

Declaration

I hereby declare that I have composed this paper independently. All passages that are quoted or paraphrased from published or unpublished works of others or from the author's own work are clearly indicated as such. All sources and aids used for this paper are specified. This paper has not been submitted in the same or similar form to any other examination authority.

Cologne, 03. July 2024
Location, Date


Signature